FRS: Topic-Based Reporting Hierarchy with Scoped Attributes

1. Feature Overview

Feature Name: Advanced Topic-Based Reporting & Approval Matrix

Objective:
Enable employees to have **different reporting persons** for different business topics (e.g., Sales Discount Approval, Expense Claims, Booking Approval, etc.), with the ability to define **scope-specific rules** and **topic-specific attributes**.

This system will work **alongside** the existing default reporting structure (`reporting_manager_code`).

---

2. Business Context & Problem Statement

Currently, the system only supports one default reporting manager per employee. However, real business scenarios require:

- Different approvers for different processes (topics).
- Approvers that change based on **organizational scope** (Branch, Location, Segment, SubSegment, etc.).
- Different data points to be captured depending on the topic (e.g., Bargain Power for discounts, Thresholds for expenses).

---

3. Key Entities

Entity	Purpose	Type
Reporting Topic	Master definition of a business topic	Master
Employee Topic Reporter	Actual reporting rule for a topic + scopes	Transactional
Employee	Existing employee record	Master
Default Reporting	Existing `reporting_manager_code`	Existing

---

4. Functional Requirements

FR-1: Topic Master

- System must have a **Topic Master** to define all reportable topics.
- Each topic must define:
  - `code` (unique, uppercase)
  - `name`
  - `description`
  - `required_attributes` (JSON) – Defines what custom fields are mandatory for that topic along with validation rules.
- New reporting rules for a topic must validate against the topic's required attributes.

Example Topic Definition:

```
{
  "code": "SALES_DISCOUNT",
  "name": "Sales Discount Approval",
  "description": "Approval for additional discount beyond standard policy",
  "required_attributes": {
    "bargain_power": {"type": "integer", "min": 0, "max": 10, "required": true},
    "max_od_discount": {"type": "decimal", "min": 0, "max": 15, "required": true}
  }
}
```

FR-2: Topic-Based Reporting Rules

- An employee can have **multiple reporting rules** for different topics.
- Each rule consists of:
  - Topic
  - Reporting To (Employee Code)
  - Scopes (with hierarchical dependency)
  - Attributes (JSON – topic specific values)
  - Priority (for future use)
  - Active status

FR-3: Scope Handling (Strict & Hierarchical)

Supported scopes with dependency:

Scope	Depends On	Default Value	Selection Behavior

Branch	-	ALL	Free selection
Location	Branch	ALL	Only locations of selected branch
Segment	-	ALL	Free selection
SubSegment	Segment	ALL	Only sub-segments of selected segment
Model	SubSegment	ALL	Only models of selected sub-segment
Vertical	-	ALL	Free selection
Department	-	ALL	Free selection
Division	Department	ALL	Only divisions of selected department

- If any scope is **ALL**, it means the rule is applicable across all values of that scope.
- **Conflict Resolution**: Strict exact match. If multiple rules match, the system should either pick the most specific match or return all matches for the caller to decide.

FR-4: Resolution Logic (Priority Order)

When resolving reporting person for a topic:

1. Search in **Topic-Based Reporting Rules** for the given employee + topic + current scopes.
2. If **no matching rule** is found Fall back to **Default Reporting Manager** (`reporting_manager_code`).
3. If topic itself does not exist in Topic Master Use default reporting.

FR-5: History Tracking

- Every change in `employee_topic_reporters` (create/update/delete) must be logged in `employee_topic_reporter_histories` table.
- Track: `old_value` , `new_value` , `changed_by` , `changed_at` , `reason` (optional).

FR-6: Import Support

- The existing `StandaloneUsersImport` (or a new dedicated importer) should support importing topic-based reporting rules.
- Validation must happen against Topic Master's `required_attributes` .

---

5. Use Case Examples

Use Case 1: Sales Discount Approval (Scoped)

Employee **BMPL-0297** should report to:

- **Person A (BMPL-0046)** for:
    - Segment = PV
    - SubSegment = XUV
    - Location = Bikaner (under Bikaner Branch)
    - Attributes: `bargain_power = 5`, `max_od_discount = 8`
- **Person B (BMPL-0085)** for:
    - Segment = LMM
    - Location = Ratangarh (under Churu Branch)
    - Attributes: `bargain_power = 3`, `max_od_discount = 6`

Use Case 2: Expense Claim

For topic `EXPENSE_CLAIM`, the same employee reports to a different person with different attributes (`threshold`, `upper_threshold`).

Use Case 3: No Specific Rule

If no rule exists for topic `BOOKING_APPROVAL` for an employee, the system should automatically use the default `reporting_manager_code`.

---

6. Proposed Data Model

Table 1: `reporting_topics`

Column	Type	Notes
id	bigint	PK
code	string	Unique, uppercase
name	string	-
description	text	Nullable
required_attributes	json	Schema + validation rules
is_active	boolean	Default true
created_by, updated_by	bigint	-

timestamps	-	-

Table 2: `employee_topic_reporters`

Column	Type	Notes
id	bigint	PK
employee_code	string	Indexed
topic_code	string	FK to reporting_topics.code
reporting_to_code	string	Employee code
scopes	json	Branch, Location, Segment, etc.
attributes	json	Topic specific values
priority	integer	Default 0
is_active	boolean	Default true
created_by, updated_by	bigint	-
timestamps	-	-

Table 3: `employee_topic_reporter_histories`

(For audit trail)

---

7. Non-Functional Requirements

- Scopes should support **cascading dropdowns** in UI later.
- Resolution logic should be efficient (preferably cached where possible).
- Strict validation on import and CRUD.
- History must be immutable.